

Report for Laboratory 4: Finite State Machine and Open-Loop Control

Julian Soh

Department of Mechanical Engineering, Saint Martin's University
ME/EE 477—Embedded Real-Time Computing for Mechanical Engineers

June 7, 2026

Abstract. Writing a program to control and monitor the speed of a DC motor implemented using a Finite State Machine (FSM). A myRIO target computer will drive the motor with pulse modulation on a DIO channel configured as a digital output. The digital signal will be amplified by an analog current amplifier. Observe the waveform generated by the FSM on an oscilloscope.

1 Introduction

We built a Finite State Machine (FSM) to control the speed of a DC motor by using a controllable signal to an amplifier that will in turn draw from a larger power source in order to produce controllable power to drive a DC motor, as shown in Figure 1.

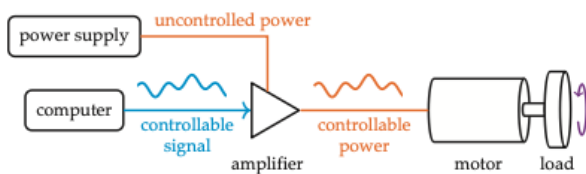


Figure 1: Typical motor-driving configuration using a signal to generate controllable power. [Picone et al. \(2024\)](#).

The objectives of this lab are:

- To implement a finite state machine control algorithm.
- To understand pulse-modulation driving of a DC motor.
- To use instruction timing to produce a calibrated delay
- To become familiar with quadrature encoders

2 Materials and Methods

2.1 Materials

The materials used in this experiment were:

- Windows 10 computer with myRIO support for C and Eclipse IDE.
- NI myRIO-1900
- Rigol DHO914S Digital Oscilloscope with 25Mhz function generator
- Cables:
 - BNC with alligator clips
 - jumper cables

2.2 Schematic of the equipment setup

The schematic of the setup is shown in Figure 2. It shows the pins that are connected to the amplifier, buttons on the

circuit board (stop and print), and the encoder.

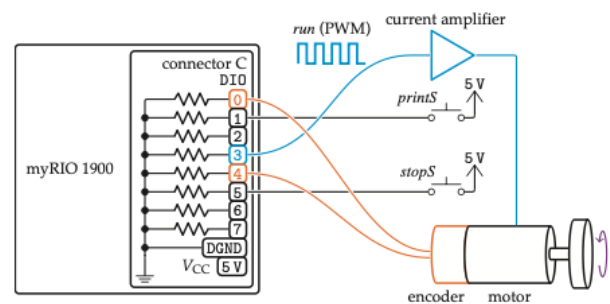


Figure 2: Schematic of connection between the myRIO and the motor control circuit. [Picone et al. \(2024\)](#).

2.3 Pulse-Width Modulation (PWM)

A motor spins when current flows through its coils, thereby creating a magnetic field that produces torque and therefore rotates the shaft.

More current results in more torque. An efficient way of control motor speed is by applying full voltage but only part of the time. The N and M values in this lab represents the pulse width and period, respectively.

Example: N = 5, M=3

HIGH HIGH HIGH LOW LOW.

The motor spins smoothly and does not jerk ON and OFF because of motor physics. Motor windings resist rapid current changes due to coil inductance so it does not stop instantly when power goes off. Therefore, the motor experiences smooth continuous torque.

The motor behaves as if it is getting

$$V_{\text{avg}} = \left(\frac{N}{M} \right) V_{\text{max}}$$

Example: For a 5V system, N=5, M=3:

$$V_{\text{avg}} = \left(\frac{3}{5} \right) 5V = 0.6 \times 5V = 3.00V$$

2.4 Finite State Machine Implementation

The Finite State Machine (FSM) implements the state transition logic depicted in Figure 3.

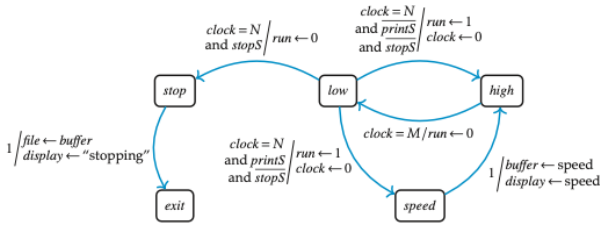


Figure 3: State Transition Diagram for the Finite State Machine. Picone et al. (2024).

Instruction Type	Mnemonic	Clock Cycles
Load/store registers	LDR, STR	2
Branch	BNE, B, BX	0
Stack	PUSH, LDMFD	2
Arithmetic	ADD, SUB, CMP	1
Move	MOV	1
No operation	NOP	1

Table 1: Opcode execution times (CPU cycles) for instructions used in wait()

3 Laboratory Exploration and Evaluation

The complete source code for the lab is available on GitHub at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab4/main.c.

A video demonstration of the lab is available at https://github.com/juliansoh/ME477/blob/main/Reports/Lab4/videos/IMG_9961.MOV.

3.1 Problem L4.1

Determine the exact number of clock cycles for the wait() function of listing 4.2 to execute, accounting for all instructions (use table 4.6). From that, calculate the delay interval in milliseconds. A spreadsheet is convenient for making this calculation.

The mapping of the wait() function to assembly code is shown in Figure 4. The loop in the wait() function is highlighted in the dotted box and is the part of the code that executes 417000 times.

The setup overhead of the wait() function only runs once, as does the loop cleanup after $i == 0$.

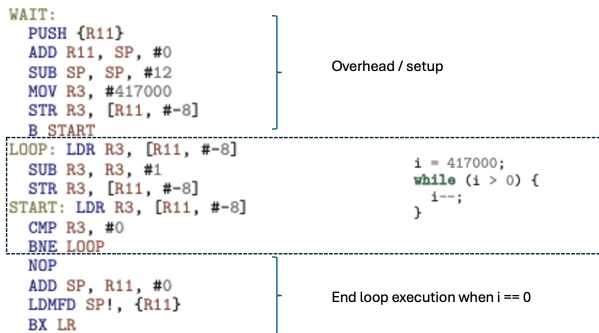


Figure 4: Mapping the wait() to assembly code.

PUSH : 2

ADD : 1

SUB : 1

MOV : 1

STR : 2

B : 0

Total setup cycles = 2 + 1 + 1 + 1 + 2 = 7

Loop iteration Each iteration of the loop executes the following instructions:

LDR : 2 cycles

SUB : 1 cycle

STR : 2 cycles

LDR : 2 cycles

CMP : 1 cycle

BNE : 0 cycles

Total cycles per loop iteration:

2 + 1 + 2 + 2 + 1 + 0 = 8 cycles

The loop executes 417000 times, so total loop cycles:

417000 × 8 = 3,336,000 cycles

Exit instructions:

NOP : 1

ADD : 1

LDMFD : 2

BX : 0

Total exit cycles = 1 + 1 + 2 = 4

Therefore, the total clock cycles for wait() is:

Total cycles = 3,336,000 + 7 + 4 = 3,336,011 cycles

Time Calculation The processor clock frequency is:

$f = 667 \text{ MHz}$

Function overhead/setup

Clock period:

$$T = \frac{1}{667 \times 10^6}$$

Total execution time:

$$t = \frac{3,336,011}{667 \times 10^6}$$

$$t \approx 0.00500 \text{ seconds} = 5.00 \text{ ms}$$

3.2 Problem L4.2

Examine the circuit board attached to connector C of the myRIO. The normally open push-button switches, when closed, connect DIO channels 1 and 5 to $V_{cc} = 5 \text{ V}$ when pressed, as shown in figure 4.19. Note: These channels have pull-down resistors. Use the oscilloscope to view the waveform produced by your program. For example, use $N = 5$, $M = 3$.

From the oscilloscope, we observed the PWM waveform as shown in Figure 5.

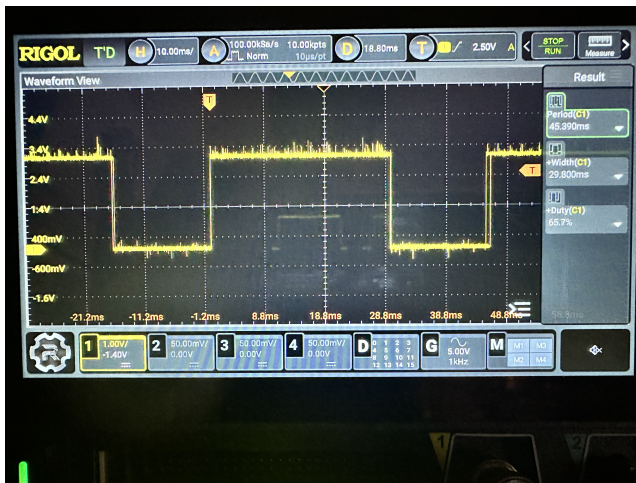


Figure 5: Image of the PWM waveform from the Oscilloscope.

3.3 Problem L4.3

Compare your program's printed output (on the LCD) to the steady state motor speed as measured with an optical tachometer.

3.4 Problem L4.4

Use the oscilloscope to view the PWM waveform produced by your program, and to measure the actual length of a BTI. Is it what you expect? If not, why not?

Refer to Figure 5 again and note the measurements taken on the waveform were:

- Period: 46 ms (N)

- Width (high): 30 ms (M)
- Duty cycle: 65%
- $M/N = 30/46 = 0.65$

We entered $N=5$ and $M=3$, so the expected duty cycle is $3/5 = 0.6$ or 60%. The actual duty cycle is 65%, which is slightly higher than expected. This could be due to timing inaccuracies in the program or the oscilloscope measurements.

3.5 Problem L4.5

Repeat lab problem L4.4 while printing the speed (pressing the switch). What does the oscilloscope show has happened to the length of the BTI? What's going on!?

When the print button is pressed, the oscilloscope shows that the length of the BTI (Base Time Interval) has changed. The new measured values were:

- Period: 56 ms (N)
- Width (high): 46 ms (M)
- Duty cycle: 81%
- $M/N = 46/56 = 0.82$

3.6 Problem L4.6

Describe how you made the measurements of lab problems L4.4 and L4.5 and discuss any limitations in the accuracy of the timing of the output. In lab exercise 6, we will find ways of overcoming this limitation.

Refer to Figure 5 that shows the screen of the oscilloscope. The measurements taken are shown to the right of the waveform: Period, +Width, and +Duty.

These measurements were taken by using the oscilloscope's measurement tools and selecting these parameters from the oscilloscope's menu.

The limitations of using these tools is that the measurements are taken in real time and may be subject to noise and other sources of error. We could just take the average of the measurements to improve accuracy. Nevertheless, this approach is more accurate than manual measurements or visual inspection.

3.7 Problem L4.7

After you have your code running as described above, try this: Record the velocity step response of the DC motor, save it to a file, and plot it in MATLAB. Here's how:

3.8 Problem L4.8

You may have noticed that when $M = 1$, the FSM does not function as desired. Address the following issues:

1. What is wrong?
2. How would modifying the state transition diagram correct this problem?
3. How would you modify the state transition table?

4. Modify your program to correct the $M = 1$ case. Test the result.

3.9 Problem L4.9

A programmer has written following code in an attempt to initialize the encoder.

```
1 MyRio_Encoder *encC0;  
2 EncoderC_initialize(myrio_session, encC0);
```

It's incorrect. What issues could this code cause, and why?

The declaration initializes a pointer to the `MyRio_Encoder` structure but does not allocate memory for the structure itself. Therefore, when the code attempts to read or write to this memory location, it will access invalid or nonexistent memory.

Instead, the `MyRio_Encoder` structure should be declared. The address operator can then be used to pass a pointer to the `MyRio_Encoder` structure. See page 230 of [Picone et al. \(2024\)](#).

4 Author Contributions

There is only one author for this report and lab.

References

Rico A. R. Picone, Joseph L. Garbini, and Cameron N. Devine. *An introduction to real-time computing for mechanical engineers*. The MIT Press, 2024.